Memory Management – Laboratory 4

Evaluating TLBs, Multi-Level Page Tables and Page Replacement on SPEC-style Traces

Skanda P (CS24BT045) N Suhas (CS24BT019)

1 Overview and Methodology

We built a trace-driven memory-virtualization simulator in C++17 (`main.cpp`, built with the supplied `Makefile` at `-O3 -march=native`). It is driven over ten benchmark traces (blender, fotonik3d, gcc, imagick, lbm, mcf, perlbench, x264, xalancbmk, xz). `run_experiments.sh` runs every configuration for every trace and writes `results.csv`; all numbers and plots in this report come from that file (no simulation was re-run while writing the report).

System model (from the assignment). 32-bit virtual addresses (the lower 32 bits of each trace address), 4 KiB pages (12 offset bits, 20-bit VPN, 2^{20} virtual pages); a 128-entry, fully associative, true-LRU TLB that starts empty; TLB access = 1 cycle; memory access = 200 cycles; no caches; a disk access (page replacement only) = 20,000,000 cycles.

Cycle accounting. Every trace line (instruction or data, read or write) is one request and is charged as follows.

- *No TLB:* (page-table walk) + 200 for the access itself. The walk is 200 cycles for the single-level table and $2 \times 200 = 400$ for the two-level table. So a request costs 400 (single) or 600 (two-level) cycles, and total cycles are exactly $400N$ and $600N$ for N requests; the results confirm this for every benchmark.
- *With TLB:* 1 cycle always; on a hit add 200 for the access; on a miss add the walk, install the entry, then add 200.
- *Replacement (Part IV):* when a page must evict another page, add 20M cycles. First-touch faults are free, as the assignment allows.

Design summary.

- **Single-level table:** a flat array of 2^{20} 32-bit entries indexed by VPN (4 MiB with 4-byte entries).
- **Two-level table:** 10-bit L1 index (bits 31–22), 10-bit L2 index (bits 21–12), 12-bit offset. The 1024-entry L1 directory is always allocated; a 1024-entry L2 table is allocated lazily the first time any page in its 4 MiB region is touched.
- **Free-frame management (our scheme):** a sequential *bump allocator*. All frames start free; the k-th distinct page ever touched gets frame $k-1$. A page that is touched again keeps its frame. Since frame numbers depend only on the order of first touches, the single-level and two-level simulators allocate identical frames. When physical memory is bounded and full (Part IV), the frame of the *least recently used* page is evicted and reused directly for the incoming page.
- **LRU for physical frames:** an $O(1)$ doubly-linked list over frames, updated on every access (including TLB hits), giving exact LRU. On eviction the victim's page-table entry is cleared and its TLB entry (if any) is invalidated.

- **TLB:** 128 slots with an exact-LRU linked list. For simulator speed, a direct-indexed table maps VPN to slot, which gives $O(1)$ lookup. This changes only simulator speed, not the modelled behaviour (still fully associative, true LRU).
- **Hash:** 64-bit FNV-1a-style update $H \leftarrow (H \oplus \mathit{frame}) \times 1099511628211 \pmod{2^{64}}$, applied once for every request, starting from $H_0 = 1469598103934665603$. The update is order-sensitive, so two runs have the same hash only if *every* access resolved to the same frame in the same order.
- **Page-table utilization:** (entries accessed)/(entries allocated). Single-level: distinct pages touched $/\, 2^{20}$. Two-level: (distinct pages + L1 entries in use, one per allocated L2 table) / $(1024 + 1024\times$ L2 tables).
- **Page-fault rate (Part IV):** all faults (first-touch and replacement) divided by total requests. We also report the number of replacement faults, since only those cost disk time.

2 Part I – Single-Level Page Table

Benchmark	Accesses (M)	Distinct pages	Frames used	Footprint (MiB)	Cycles (10^9)	PT utilization
blender	12.37	1,551	1,551	6.06	4.948	0.1479%
fotonik3d	12.89	1,602	1,602	6.26	5.156	0.1528%
gcc	12.47	990	990	3.87	4.989	0.0944%
imagick	11.55	96	96	0.38	4.620	0.0092%
lbm	12.84	2,131	2,131	8.32	5.137	0.2032%
mcf	12.47	1,597	1,597	6.24	4.990	0.1523%
perlbench	12.57	630	630	2.46	5.027	0.0601%
x264	11.72	234	234	0.91	4.689	0.0223%
xalancbmk	11.97	14,325	14,325	55.96	4.789	1.3661%
xz	11.85	8,037	8,037	31.39	4.741	0.7665%

Table 1: Part I: single-level page table, no TLB, unlimited frames. Every request costs $200 + 200 = 400$ cycles.

Benchmark	64-bit hash H (hex)
blender	`d98327b77cd06bdd`
fotonik3d	`d734ccdcef856525`
gcc	`ff879fd3af3d4b14`
imagick	`694458f867debd44`
lbm	`f8e1d67f185397e0`
mcf	`3d667ce5b27027a5`
perlbench	`693f48a2f9232bb0`
x264	`6c050606936f18d2`
xalancbmk	`e4b42faaf70a41ad`
xz	`a6615c047b39620d`

Table 2: Final hash for every benchmark. Identical for single-level, two-level, with and without TLB (unlimited memory).

(A) Frames used. The number of frames equals the number of distinct pages touched, because memory is unbounded and the bump allocator never reuses a frame. It ranges from only 96 frames (imagick, 0.4 MiB) to 14,325 frames (xalancbmk, about 56 MiB), so even the largest benchmark touches only 1.4% of the 2^{20} virtual pages.

(B) Hot-page hypothesis. Figure 1 plots, for each benchmark, the cumulative fraction of all accesses captured by the hottest x fraction of touched pages. The diagonal is what uniform access would give. Every curve lies far above it: the hottest 1% of pages already receive 34–99% of the accesses, and the hottest 20% receive about 91% or more in *every* benchmark (Table 3). Conversely, the colder half of the pages receives only about 0.1–4.3% of the accesses. The **hypothesis is supported**. The strength differs by benchmark:

- xz, xalancbmk, mcf, lbm and blender are extremely skewed. About 0.1–0.6% of their pages account for 90% of accesses (a handful of stack, hot-data and code pages). Their curves are nearly vertical at the origin and flat afterwards.
- gcc, perlbench, x264 and imagick are broader: roughly 10–16% of pages are needed for 90% of accesses (Figure 2). They still beat uniform access by a large margin.
- fotonik3d and imagick have a tiny dominant core followed by a long, slowly rising tail. fotonik3d gets 90.1% of its accesses from its top 1% of pages but only 95.9% from its top half, and imagick's single hottest page (1 of 96) takes 86.5%. The remaining pages are touched sparsely (e.g. streaming through arrays).

This skew is what makes a small TLB and a small physical memory effective (Parts III and IV).

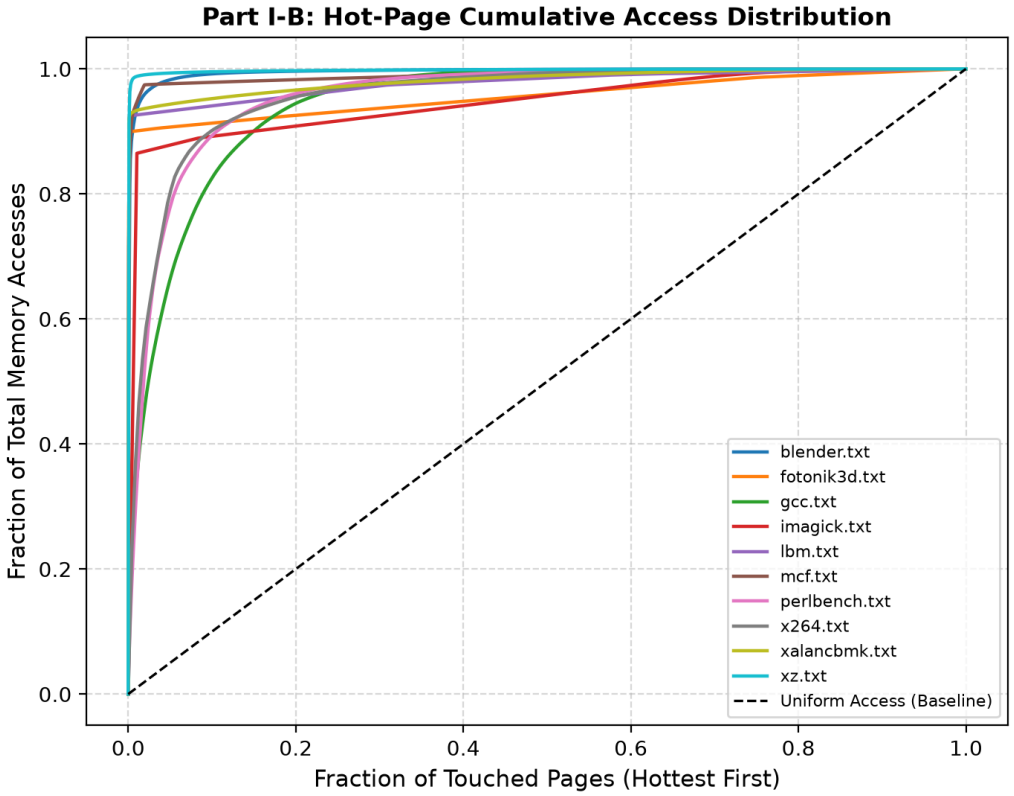


Figure 1: Hot-page cumulative access distribution (page-sorted hottest first) for all ten benchmarks.

Benchmark	top 1%	top 5%	top 10%	top 20%	top 50%	Pages for 90% of accesses
blender	93.7%	98.3%	99.2%	99.6%	99.9%	0.58% (9)
fotonik3d	90.1%	90.7%	91.3%	92.6%	95.9%	0.50% (8)
gcc	34.3%	66.6%	82.5%	94.5%	99.8%	14.95% (148)
imagick	86.5%	87.9%	89.3%	91.0%	95.8%	15.62% (15)
lbm	92.7%	93.3%	94.1%	95.7%	98.6%	0.19% (4)
mcf	94.8%	97.6%	97.9%	98.3%	99.4%	0.38% (6)
perlbench	35.2%	78.2%	89.3%	96.1%	99.6%	10.63% (67)
x264	46.6%	80.8%	90.4%	95.5%	99.4%	10.26% (24)
xalancbmk	93.4%	94.4%	95.3%	96.6%	98.9%	0.10% (14)
xz	98.8%	99.4%	99.5%	99.7%	99.9%	0.10% (8)

Table 3: Share of all accesses received by the hottest x% of the touched pages (single-level run; values read off the 1000-point CDF, so the 1% column is approximate for tiny footprints such as imagick).

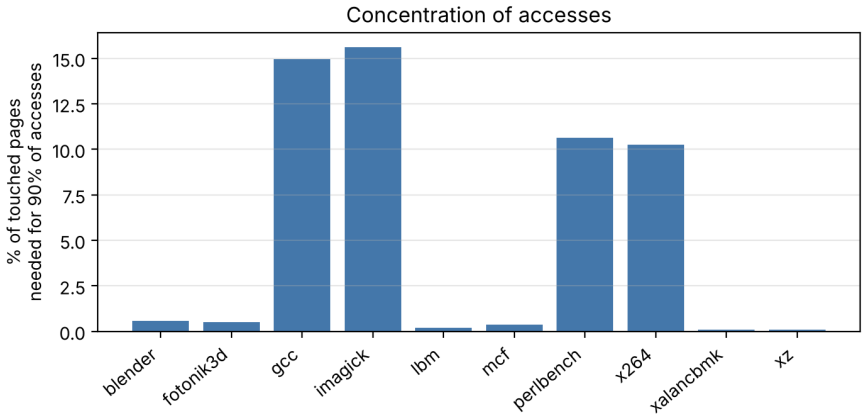


Figure 2: Fraction of touched pages that account for 90% of all accesses.

(C) Cycles. Without a TLB every request costs exactly 400 cycles, so execution time is $400N$ and is determined purely by the length of the trace (4.6–5.2 billion cycles for 11.5–12.9 million requests). The page-table walk is half of the cost: translation is as expensive as the access itself.

(D) Page-table utilization. The flat table allocates all 2^{20} entries up front but each benchmark touches at most 14,325 of them, so utilization is tiny: 0.009% (imagick) to 1.37% (xalancbmk). A flat table wastes almost all of its 4 MiB for these programs.

(E) Hash. See the hash table above. Hashes differ between benchmarks as expected; they are used in Part II to check that both page-table organisations behave identically.

3 Part II – Two-Level Page Table

Benchmark	L2 tables	Entries alloc.	Size (KiB)	PT util.	Cycles (10^9)	vs. single
blender	47	49,152	192	3.25%	7.422	1.50×
fotonik3d	14	15,360	60	10.52%	7.734	1.50×
gcc	70	72,704	284	1.46%	7.484	1.50×
imagick	5	6,144	24	1.64%	6.929	1.50×
lbm	6	7,168	28	29.81%	7.706	1.50×
mcf	13	14,336	56	11.23%	7.485	1.50×
perlbench	48	50,176	196	1.35%	7.541	1.50×
x264	11	12,288	48	1.99%	7.033	1.50×
xalancbmk	54	56,320	220	25.53%	7.183	1.50×
xz	43	45,056	176	17.93%	7.111	1.50×

Table 4: Part II: two-level page table (10+10+12 bit split), no TLB, unlimited frames. A walk costs 2×200 cycles, plus 200 for the data access. The single-level table is always 2^{20} entries = 4096 KiB.

Frames used and hash. Frames used are unchanged from Table 1 (the same 96–14,325), as frame allocation does not depend on the table structure. **The hash matches Part I exactly for all ten benchmarks** (Table 2). This is expected: the page-table organisation only changes *how* a VPN is translated, not *which* frame it maps to, and the bump allocator hands out frames in first-touch order in both simulators.

Cycles. Each walk now needs two memory accesses, so every request costs 600 cycles, exactly 1.5× the single-level cost for every benchmark.

Utilization and space. Utilization rises sharply, from 0.009–1.4% (single-level) to 1.3–29.8%, because only the L2 tables that cover touched 4 MiB regions are allocated. The page table shrinks from 4096 KiB to between 24 KiB (imagick) and 284 KiB (gcc), i.e. 0.6–6.9% of the flat table. How well the two-level table is used depends on how *clustered* the pages are: lbm packs its 2,131 pages into just 6 L2 tables (29.8% utilization), whereas gcc scatters 990 pages over 70 regions (1.5%), so each region has a mostly empty L2 table. The price is one extra memory access per translation (the 1.5× above), which is why a TLB matters for multi-level tables.

4 Part III – Adding the TLB

Benchmark	TLB hit rate	1-lvl cyc.	2-lvl cyc.	Speedup 1-lvl	Speedup 2-lvl	2-lvl ovh.
blender	99.940%	2.488	2.489	1.99×	2.98×	0.06%
fotonik3d	99.987%	2.591	2.592	1.99×	2.98×	0.01%
gcc	99.887%	2.510	2.513	1.99×	2.98×	0.11%
imagick	99.999%	2.321	2.321	1.99×	2.99×	0.00%
lbm	99.979%	2.582	2.583	1.99×	2.98×	0.02%
mcf	99.442%	2.521	2.535	1.98×	2.95×	0.55%
perlbench	99.548%	2.538	2.549	1.98×	2.96×	0.45%
x264	99.966%	2.357	2.358	1.99×	2.98×	0.03%
xalancbmk	99.867%	2.410	2.413	1.99×	2.98×	0.13%
xz	99.767%	2.388	2.393	1.99×	2.97×	0.23%

Table 5: Part III: 128-entry fully associative LRU TLB, unlimited frames. Cycle columns are in units of 10^9. Cycle columns are in units of 10^9. Speedup is relative to the same page table without a TLB; the last column (ovh.) is the extra time of the two-level table over the single-level one, both with TLB.

Hit rate. The TLB hit rate is 99.4–99.999% for all benchmarks (lowest: mcf 99.44%, perlbench 99.55%; highest: imagick 99.9992%). The hit rate is identical for single- and two-level tables because

TLB contents depend only on the VPN stream. A 128-entry TLB covers only 512 KiB, but this is enough because of the skew shown in Part I(B).

Cycles. Almost every request now costs 201 cycles (1 TLB + 200 access) instead of 400/600. The speedup is about 2× for the single-level table and about 3× for the two-level table: the deeper the walk, the more the TLB saves. With a TLB the two-level table is within 0.6% of the single-level one (0.001% for imagick, 0.55% for mcf), so it keeps its space saving at essentially no performance cost. Benchmarks with lower hit rates (mcf, perlbench) pay the largest two-level overhead, as expected, since the two-level penalty is paid only on misses.

5 Part IV – Bounded Physical Memory

We ran each benchmark with the TLB enabled and LRU replacement. The assignment suggests memory sizes of 0.5×, 0.25×, ... of the virtual memory size (2^{20} pages). We did run 0.5, 0.25, 0.125, 0.0625× (524,288, 262,144, 131,072, 65,536 frames). However, the largest footprint in our traces is 14,325 pages (1.4% of virtual memory), so *no replacement ever occurs in these runs and results are identical to the unbounded case* (we verified cycle counts, fault counts and hashes are equal). Because those sizes are uninformative for these traces, we added absolute sizes of 4096, 1024, 256, 64 and 16 frames, which span from above to far below each benchmark's working set.

Benchmark	4,096 fr.	1,024 fr.	256 fr.	64 fr.	16 fr.
blender	0.0125%	0.0151%	0.0381%	0.3699%	0.7382%
fotonik3d	0.0124%	0.0133%	0.0133%	0.0133%	0.3103%
gcc	0.0079%	0.0079%	0.0400%	0.3199%	1.9926%
imagick	0.0008%	0.0008%	0.0008%	0.0568%	0.0581%
lbm	0.0166%	0.0211%	0.0211%	0.0211%	0.0523%
mcf	0.0128%	0.0131%	0.3253%	0.7046%	1.6953%
perlbench	0.0050%	0.0050%	0.1620%	0.9100%	2.9318%
x264	0.0020%	0.0020%	0.0020%	0.0539%	0.1631%
xalancbmk	0.1250%	0.1251%	0.1252%	0.1381%	0.2082%
xz	0.0778%	0.1310%	0.2029%	0.2779%	1.0963%

Table 6: Part IV: page-fault rate (all faults, including first-touch ones, divided by total accesses) versus number of physical frames. Identical for single- and two-level tables.

Benchmark	4,096 fr.	1,024 fr.	256 fr.	64 fr.	16 fr.
blender	0	849	4,454	45,688	91,293
fotonik3d	0	693	1,461	1,653	39,983
gcc	0	0	4,738	39,835	248,533
imagick	0	0	0	6,495	6,697
lbm	0	1,680	2,448	2,640	6,700
mcf	0	614	40,327	87,836	211,461
perlbench	0	0	20,109	114,311	368,458
x264	0	0	0	6,255	19,102
xalancbmk	10,873	13,950	14,736	16,473	24,905
xz	5,121	14,503	23,792	32,873	129,920

Table 7: Part IV: number of replacement faults (each costing 20M cycles). First-touch faults are free.

Benchmark	4,096 fr.	1,024 fr.	256 fr.	64 fr.	16 fr.
blender	2.49×10^{9}	1.95×10^{10}	9.16×10^{10}	9.16×10^{11}	1.83×10^{12}
fotonik3d	2.59×10^{9}	1.65×10^{10}	3.18×10^{10}	3.57×10^{10}	8.02×10^{11}
gcc	2.51×10^{9}	2.51×10^{9}	9.73×10^{10}	7.99×10^{11}	4.97×10^{12}
imagick	2.32×10^{9}	2.32×10^{9}	2.32×10^{9}	1.32×10^{11}	1.36×10^{11}
lbm	2.58×10^{9}	3.62×10^{10}	5.15×10^{10}	5.54×10^{10}	1.37×10^{11}
mcf	2.52×10^{9}	1.48×10^{10}	8.09×10^{11}	1.76×10^{12}	4.23×10^{12}
perlbench	2.54×10^{9}	2.54×10^{9}	4.05×10^{11}	2.29×10^{12}	7.37×10^{12}
x264	2.36×10^{9}	2.36×10^{9}	2.36×10^{9}	1.27×10^{11}	3.84×10^{11}
xalancbmk	2.20×10^{11}	2.81×10^{11}	2.97×10^{11}	3.32×10^{11}	5.01×10^{11}
xz	1.05×10^{11}	2.92×10^{11}	4.78×10^{11}	6.60×10^{11}	2.60×10^{12}

Table 8: Part IV: total cycles, single-level page table + TLB.

Benchmark	4,096 fr.	1,024 fr.	256 fr.	64 fr.	16 fr.
blender	2.49×10^{9}	1.95×10^{10}	9.16×10^{10}	9.16×10^{11}	1.83×10^{12}
fotonik3d	2.59×10^{9}	1.65×10^{10}	3.18×10^{10}	3.57×10^{10}	8.02×10^{11}
gcc	2.51×10^{9}	2.51×10^{9}	9.73×10^{10}	7.99×10^{11}	4.97×10^{12}
imagick	2.32×10^{9}	2.32×10^{9}	2.32×10^{9}	1.32×10^{11}	1.36×10^{11}
lbm	2.58×10^{9}	3.62×10^{10}	5.15×10^{10}	5.54×10^{10}	1.37×10^{11}
mcf	2.54×10^{9}	1.48×10^{10}	8.09×10^{11}	1.76×10^{12}	4.23×10^{12}
perlbench	2.55×10^{9}	2.55×10^{9}	4.05×10^{11}	2.29×10^{12}	7.37×10^{12}
x264	2.36×10^{9}	2.36×10^{9}	2.36×10^{9}	1.27×10^{11}	3.84×10^{11}
xalancbmk	2.20×10^{11}	2.81×10^{11}	2.97×10^{11}	3.32×10^{11}	5.01×10^{11}
xz	1.05×10^{11}	2.92×10^{11}	4.78×10^{11}	6.60×10^{11}	2.60×10^{12}

Table 9: Part IV: total cycles, two-level page table + TLB.

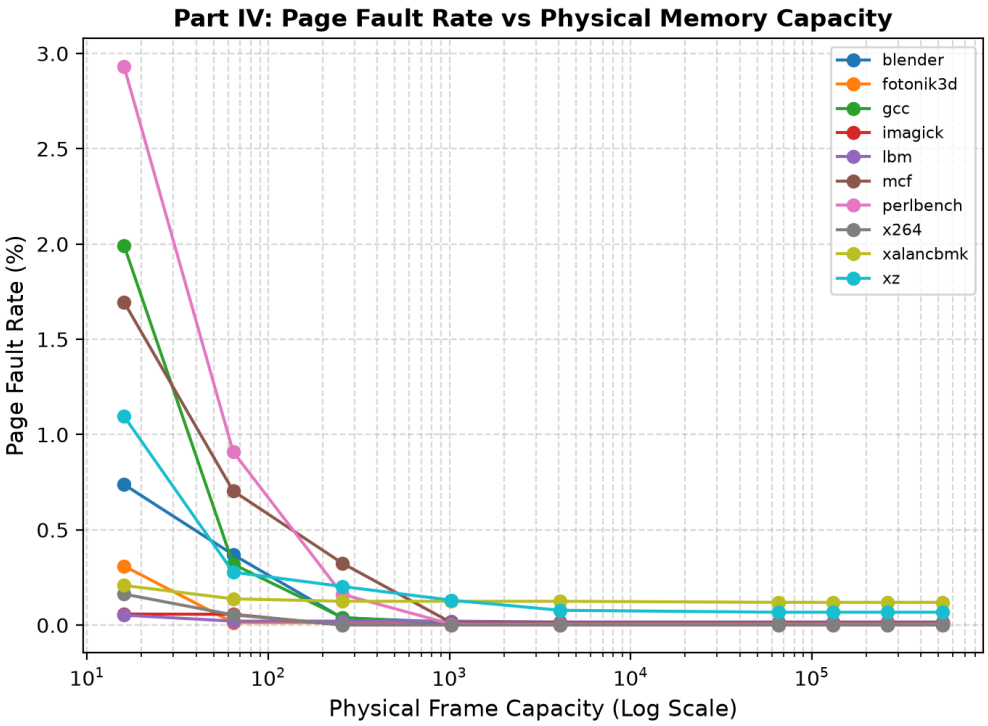


Figure 3: Page-fault rate (%) versus physical frame capacity (log scale).

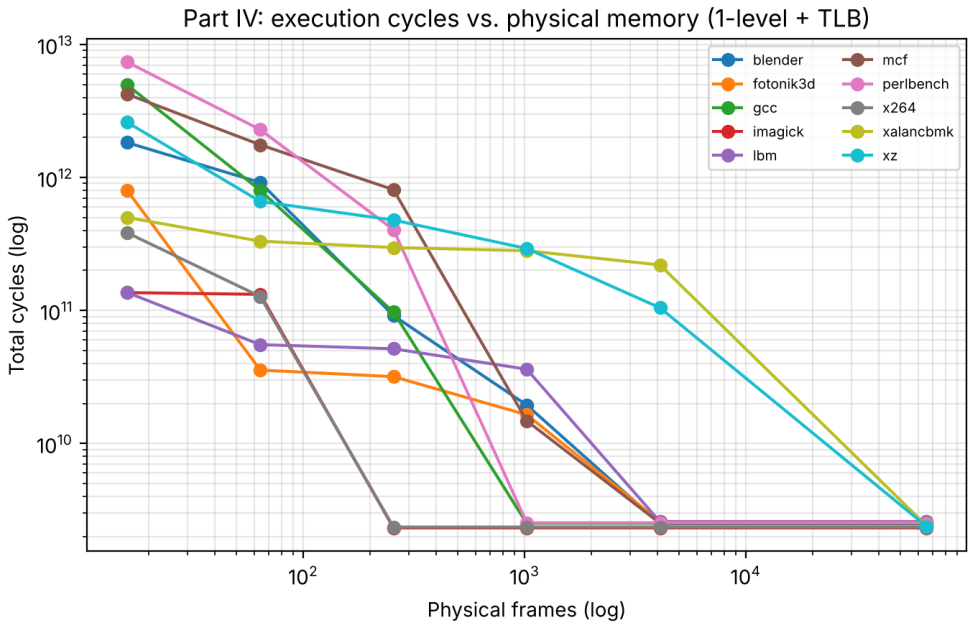


Figure 4: Total execution cycles versus number of frames (single-level + TLB), log–log. The jump away from the flat region is the disk cost.

Observations.

- **A page fault with replacement dominates everything.** One replacement costs 20M cycles, equal to about 100,000 ordinary memory accesses. For example, blender with 1024 frames has 849 replacements and $849 \times 20\text{M} = 1.698 \times 10^{10}$ cycles, exactly the increase over the unbounded run (2.49×10^9 to 1.95×10^{10}). Cycles are therefore essentially (unbounded cycles) + (replacements × 20M), with only a small extra term from TLB misses caused by invalidations. A fault rate of well under 1% is already enough to multiply run time by tens to hundreds.
- **Frame counts above the footprint change nothing.** imagick (96 pages), x264 (234), perlbench (630) and gcc (990) have zero replacements at 1024 frames (x264 also at 256; imagick also at 256), and results equal the unbounded run.
- **Working-set knees.** mcf goes from 614 replacements at 1024 frames to 40,327 at 256 (about 65×), and perlbench from none at 1024 to 20,109 at 256 even though it has 630 pages. gcc likewise jumps from none (at 1024) to 4,738 (at 256) and 39,835 (at 64). This is the signature of a working set that fits just above one size and thrashes just below it. The sharply skewed benchmarks suffer most at the smallest sizes: perlbench reaches the highest fault rate (2.93% at 16 frames, 7.4×10^{12} cycles), followed by gcc (1.99%) and mcf (1.70%).
- **Streaming benchmarks plateau.** lbm and fotonik3d have almost the same number of faults at 1024, 256 and 64 frames (2,704 and 1,717 respectively), which is only slightly more than their footprint (2,131 and 1,602). Their pages are mostly used in a streaming fashion and a small window of frames suffices; they only degrade at 16 frames (lbm 6,716 faults, fotonik3d 39,999).
- **Long-tail benchmarks degrade gradually.** xalancbmk and xz have large footprints and fault at 4096 frames already (10,873 and 5,121 replacements, cycles 2.2×10^{11} and 1.05×10^{11}), and the fault rate then grows slowly with smaller memory because their hot core is tiny but the rest of the pages are revisited sporadically (a page that was evicted is faulted in again when revisited).
- **Page table choice does not affect faults.** Replacement depends only on the page-reference

stream and LRU, so fault rates, replacement counts and hashes are identical for single- and two-level tables; cycle counts differ only by the walk cost (compare the two cycle tables).

6 Part V – Simulator Speed

Benchmark	1-lvl, no TLB	2-lvl, no TLB	1-lvl + TLB	2-lvl + TLB	M req/s (1-lvl+TLB)
blender	0.264	0.357	0.266	0.331	46.5
fotonik3d	0.287	0.308	0.293	0.373	44.0
gcc	0.275	0.279	0.289	0.334	43.2
imagick	0.220	0.245	0.227	0.281	50.9
lbm	0.307	0.312	0.304	0.387	42.2
mcf	0.287	0.276	0.294	0.336	42.4
perlbench	0.282	0.288	0.301	0.335	41.8
x264	0.239	0.244	0.241	0.280	48.6
xalancbmk	0.241	0.250	0.245	0.285	48.9
xz	0.245	0.255	0.250	0.297	47.4

Table 10: Part V: wall-clock simulation time in seconds (unlimited frames; measured inside the program around the trace loop, i.e. parsing + simulation).

All ten traces (11.5–12.9 million requests each) are simulated in about 0.22–0.39 seconds in every configuration, i.e. roughly 42–51 million requests per second for the single-level + TLB configuration. The two-level configurations are slower (11–27% with a TLB) because each TLB miss chases an extra pointer. Times come from a single run per configuration and the CPU model was not recorded in the results, so absolute numbers depend on the machine and small differences between runs are noise.

Optimisations used.

- The trace is read with large (4 MiB) `fread` chunks and split with `memchr`; a hand-written parser converts the address and flags directly from the buffer (no `scanf`/`iostream`, no per-line allocation).
- The simulation loop is templated on the page-table type and on whether the TLB is enabled, so no runtime branching on configuration remains in the hot loop.
- Both the TLB and physical-memory LRU are $O(1)$ doubly-linked lists over array indices, and the TLB uses a direct VPN→slot table instead of scanning 128 entries per request.
- Single-level table, per-page access counters and VPN-indexed structures are flat arrays; frames are allocated by a bump pointer; the hash is one XOR and one multiply per request.
- The two-level table allocates L2 tables lazily, so memory touched is proportional to the footprint.

7 Conclusions

- Memory accesses in all ten benchmarks are highly skewed: 10–20% of pages (and for several benchmarks well under 1%) absorb 82–99% of the requests, which is why a 128-entry TLB achieves >99.4% hit rate.
- A two-level table cuts page-table space from 4 MiB to 24–284 KiB (utilization up from ≤1.4% to as high as 29.8%) while computing exactly the same mapping (identical hash); without a TLB it is 50% slower, with a TLB the penalty is <0.6%.

- The TLB gives about 2× (single-level) and 3× (two-level) speedups.
- Once physical memory is smaller than the working set, execution time is dominated by the 20M-cycle replacements; a fault rate of a few tenths of a percent already costs two orders of magnitude in cycles. The working-set knee differs per benchmark (about 256–1024 frames for gcc, mcf and perlbench; thousands of frames for xalancbmk and xz).

Reproducing. `make && ./run_experiments.sh <trace_dir> <results_dir>` builds the simulator and generates `results.csv`, the per-benchmark CDF files and the plots. A single run is, e.g., `./main --trace gcc.txt --pt two --tlb --frames 256 --csv`.